

Weekly hour, plus the final session

Office hours and capstone demo day

Unscripted by design. This sheet exists so the hour has a shape when nobody brings anything, and so the last one runs to time.

First ten minutes

Collect before solving

Goal Every blocker in the room is written down before any of them is worked on.

Say

- Ask for problems in one line each and write them all up before touching any of them.
- Look for the repeated one. When three people are stuck the same way, that is a teaching gap rather than three bugs.
- Say the order out loud so nobody is waiting without knowing whether their turn is coming.

Run Nothing. A list on screen.

Room does Contributes one line each, including those who came only to listen.

Ask *Anything here that more than one of you recognises?*

As they come up

The four that always appear

Goal Recurring problems get a fast answer so the hour goes to the interesting ones.

- Say**
- A new workspace with no dependencies installed. Answer with caching and a preparation command rather than a longer setup step.
 - Context that never arrives. Almost always a path the ownership graph does not claim, so run the graph check first.
 - A check that passes when it should not. Apply the reverting proof before debugging anything else.
 - A run that never ends. Look for the missing limit rather than the slow model.
-

Run

```
$ node steps/02-execution/demo.ts selftest  
$ node steps/01-boundary/demo.ts graph
```

Room does Runs the same two commands on their own project before describing the symptom.

Ask *What did the self-check say? Read it out before we theorise.*

Final session only

Demonstrations

Goal Everyone shows their own item moving, and gets one useful piece of feedback from the group.

- Say**
- Give each person a fixed slot and hold it, so the last presenter gets the same time as the first.
 - Ask the same two questions of everyone: which layer took the longest to get right, and what did it refuse to do.
 - Direct the feedback towards what the group can use rather than towards praise.
-

Run Their machines, their repositories. Yours stays closed.

- Room does**
- Shows an item becoming a reviewed request on their own codebase.
 - Names one thing they would build differently.
-

Ask *What will you take back to your team on Monday, in one sentence?*

0:20 to 0:40

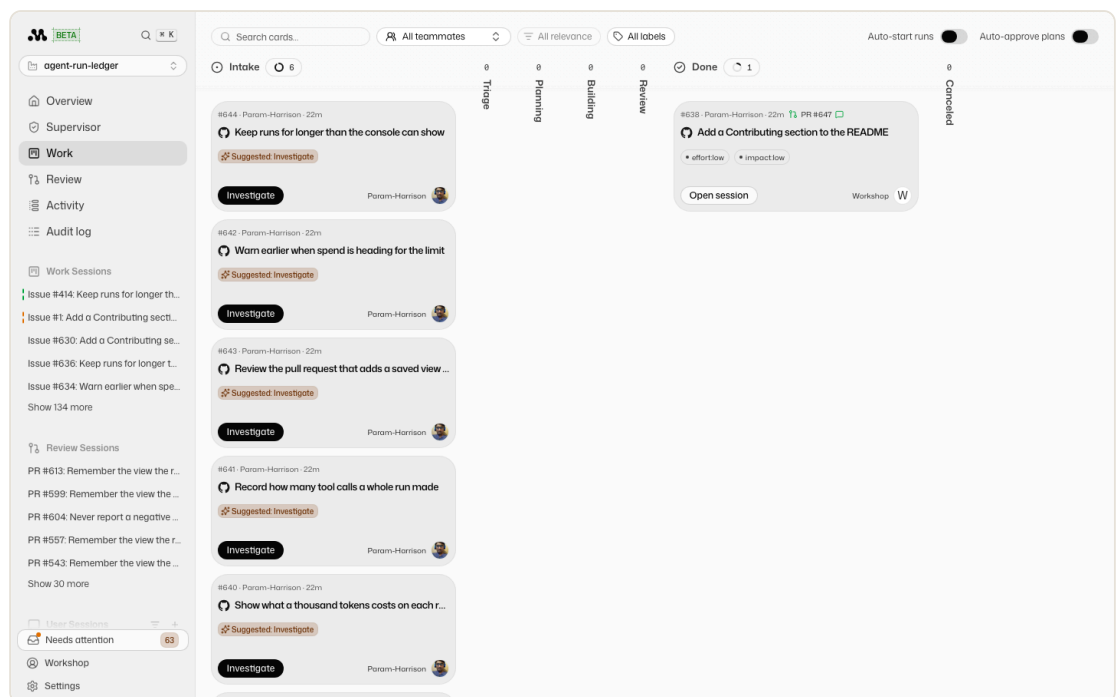
Reading somebody else's evidence

Goal Everyone has read one run they did not perform and formed a view without being told the story.

- Say**
- Bring up a capstone board and review it cold, with no narration from its author.
 - Judge the evidence rather than the outcome, because a green run with no proof is the thing this whole course argues against.
 - Let the author respond only after the room has committed to a verdict.

Run

```
$ make status
$ make score
```



Pull requests on their own board, with their own phases.

Show on the board

Review to somebody else's board to a pull request to the verdict

Judge the evidence, not the outcome

A green run with no proof is the thing this course argues against. Ask what would have to be true for this verdict to be wrong, before the author says anything.

The author speaks last

Let the room commit to a view first. A narrated demo teaches the narrator, and everyone else watches somebody else be competent.

One number each, and what it cost

Every capstone reports a coverage figure and a score. Ask which of the two they trust and why, because the gap between them is the only interesting part.

Room does

- States a verdict on a run they did not make.
- Names the one piece of evidence that would have changed it.

Ask

What did you believe about this run before the author spoke, and what changed after?

Concepts: records, endings.